# Defensive programming

**Defensive programming** is a form of defensive design intended to develop programs that are capable of detecting potential security abnormalities and make predetermined responses.[1] It ensures the continuing function of a piece of software under unforeseen circumstances. Defensive programming practices are often used where high availability, safety, or security is needed.

Defensive programming is an approach to improve software and source code, in terms of:

- General quality – reducing the number of software bugs and problems.
- Making the source code comprehensible – the source code should be readable and understandable so it is approved in a code audit.
- Making the software behave in a predictable manner despite unexpected inputs or user actions.

Overly defensive programming, however, may safeguard against errors that will never be encountered, thus incurring run-time and maintenance costs.

## Secure programming

Secure programming is the subset of defensive programming concerned with computer security. Security is the concern, not necessarily safety or availability (the software may be allowed to fail in certain ways). As with all kinds of defensive programming, avoiding bugs is a primary objective; however, the motivation is not as much to reduce the likelihood of failure in normal operation (as if safety were the concern), but to reduce the attack surface – the programmer must assume that the software might be misused actively to reveal bugs, and that bugs could be exploited maliciously.

```
int risky_programming(char *input) {
  char str[1000];

  // ...

  strcpy(str, input);  // Copy input.

  // ...
}
```

The function will result in undefined behavior when the input is over 1000 characters. Some programmers may not feel that this is a problem, supposing that no user will enter such a long input. This particular bug demonstrates a vulnerability which enables buffer overflow exploits. Here is a solution to this example:

```
int secure_programming(char *input) {
  char str[1000+1];  // One more for the null character.

  // ...

  // Copy input without exceeding the length of the destination.
```

```
  strncpy(str, input, sizeof(str));

  // If strlen(input) >= sizeof(str) then strncpy won't null terminate.
  // We counter this by always setting the last character in the buffer to NUL,
  // effectively cropping the string to the maximum length we can handle.
  // One can also decide to explicitly abort the program if strlen(input) is
  // too long.
  str[sizeof(str) - 1] = '\0';

  // ...
}
```

# Offensive programming

Offensive programming is a category of defensive programming, with the added emphasis that certain errors should *not* be handled defensively. In this practice, only errors from outside the program's control are to be handled (such as user input); the software itself, as well as data from within the program's line of defense, are to be trusted in this methodology.

## Trusting internal data validity

### Overly defensive programming

```
const char* trafficlight_colorname(enum traffic_light_color c) {
    switch (c) {
        case TRAFFICLIGHT_RED:    return "red";
        case TRAFFICLIGHT_YELLOW: return "yellow";
        case TRAFFICLIGHT_GREEN:  return "green";
    }
    return "black"; // To be handled as a dead traffic light.
}
```

### Offensive programming

```
const char* trafficlight_colorname(enum traffic_light_color c) {
    switch (c) {
        case TRAFFICLIGHT_RED:    return "red";
        case TRAFFICLIGHT_YELLOW: return "yellow";
        case TRAFFICLIGHT_GREEN:  return "green";
    }
    assert(0); // Assert that this section is unreachable.
}
```

## Trusting software components

### Overly defensive programming

```
if (is_legacy_compatible(user_config)) {
    // Strategy: Don't trust that the new code behaves the same
    old_code(user_config);
} else {
    // Fallback: Don't trust that the new code handles the same cases
    if (new_code(user_config) != OK) {
        old_code(user_config);
    }
}
```

### Offensive programming

```
// Expect that the new code has no new bugs
if (new_code(user_config) != OK) {
    // Loudly report and abruptly terminate program to get proper attention
    report_error("Something went very wrong");
    exit(-1);
}
```

# Techniques

Here are some defensive programming techniques:

## Intelligent source code reuse

If existing code is tested and known to work, reusing it may reduce the chance of bugs being introduced.

However, reusing code is not *always* good practice. Reuse of existing code, especially when widely distributed, can allow for exploits to be created that target a wider audience than would otherwise be possible and brings with it all the security and vulnerabilities of the reused code.

When considering using existing source code, a quick review of the modules(sub-sections such as classes or functions) will help eliminate or make the developer aware of any potential vulnerabilities and ensure it is suitable to use in the project.

### Legacy problems

Before reusing old source code, libraries, APIs, configurations and so forth, it must be considered if the old work is valid for reuse, or if it is likely to be prone to legacy problems.

Legacy problems are problems inherent when old designs are expected to work with today's requirements, especially when the old designs were not developed or tested with those requirements in mind.

Many software products have experienced problems with old legacy source code; for example:

- Legacy code may not have been designed under a defensive programming initiative, and might therefore be of much lower quality than newly designed source code.
- Legacy code may have been written and tested under conditions which no longer apply. The old quality assurance tests may have no validity any more.
    - **Example 1**: legacy code may have been designed for ASCII input but now the input is UTF-8.
    - **Example 2**: legacy code may have been compiled and tested on 32-bit architectures, but when compiled on 64-bit architectures, new arithmetic problems may occur (e.g., invalid signedness tests, invalid type casts, etc.).
    - **Example 3**: legacy code may have been targeted for offline machines, but becomes vulnerable once network connectivity is added.
- Legacy code is not written with new problems in mind. For example, source code written in 1990 is likely to be prone to many code injection vulnerabilities, because most such problems were not widely understood at that time.

Notable examples of the legacy problem:

- BIND 9, presented by Paul Vixie and David Conrad as "BINDv9 is a complete rewrite", "Security was a key consideration in design",[2] naming security, robustness, scalability and new protocols as key concerns for rewriting old legacy code.
- Microsoft Windows suffered from "the" Windows Metafile vulnerability and other exploits related to the WMF format. Microsoft Security Response Center describes the WMF-features as *"Around 1990, WMF support was added... This was a different time in the security landscape... were all completely trusted"*,[3] not being developed under the security initiatives at Microsoft.
- Oracle is combating legacy problems, such as old source code written without addressing concerns of SQL injection and privilege escalation, resulting in many security vulnerabilities which have taken time to fix and also generated incomplete fixes. This has given rise to heavy criticism from security experts such as David Litchfield, Alexander Kornbrust, Cesar Cerrudo.[4][5][6] An additional criticism is that default installations (largely a legacy from old versions) are not aligned with their own security recommendations, such as Oracle Database Security Checklist, which is hard to amend as many applications require the less secure legacy settings to function correctly.

## Canonicalization

Malicious users are likely to invent new kinds of representations of incorrect data. For example, if a program attempts to reject accessing the file "/etc/passwd", a cracker might pass another variant of this file name, like "/etc/./passwd". Canonicalization libraries can be employed to avoid bugs due to non-canonical input.

## Low tolerance against "potential" bugs

Assume that code constructs that appear to be problem prone (similar to known vulnerabilities, etc.) are bugs and potential security flaws. The basic rule of thumb is: "I'm not aware of all types of security exploits. I must protect against those I *do* know of and then I must be proactive!".

## Other ways of securing code

- One of the most common problems is unchecked use of constant-size or pre-allocated structures for dynamic-size data such as inputs to the program (the buffer overflow problem). This is especially common for string data in C. C library functions like `gets` should never be used since the maximum size of the input buffer is not passed as an argument. C library functions like `scanf` can be used safely, but require the programmer to take care with the selection of safe format strings, by sanitizing it before using it.
- Encrypt/authenticate all important data transmitted over networks. Do not attempt to implement your own encryption scheme, use a proven one instead. Message checking with a hash or similar technology will also help secure data sent over a network.

### The three rules of data security

- All data is important until proven otherwise.
- All data is tainted until proven otherwise.
- All code is insecure until proven otherwise.

- You cannot prove the security of any code in userland, or, more commonly known as: *"never trust the client"*.

These three rules about data security describe how to handle any data, internally or externally sourced:

**All data is important until proven otherwise** - means that all data must be verified as garbage before being destroyed.

**All data is tainted until proven otherwise** - means that all data must be handled in a way that does not expose the rest of the runtime environment without verifying integrity.

**All code is insecure until proven otherwise** - while a slight misnomer, does a good job reminding us to never assume our code is secure as bugs or undefined behavior may expose the project or system to attacks such as common SQL injection attacks.

### More Information

- If data is to be checked for correctness, verify that it is correct, not that it is incorrect.
- Design by contract
- Assertions (also called **assertive programming**)

## See also

- Computer security

## References


1. Boulanger, Jean-Louis (2016-01-01), "6 - Technique to Manage Software Safety" (https://www.sciencedirect.com/science/article/pii/B9781785481178500064), in Boulanger, Jean-Louis (ed.), *Certifiable Software Applications 1*, Elsevier, pp. 125–156, ISBN 978-1-78548-117-8, retrieved 2022-09-02
2. "fogo archive: Paul Vixie and David Conrad on BINDv9 and Internet Security by Gerald Oskoboiny" (http://impressive.net/archives/fogo/20001005080818.O15286@impressive.net). *impressive.net*. Retrieved 2018-10-27.
3. "Looking at the WMF issue, how did it get there?" (https://web.archive.org/web/20060324152626/http://blogs.technet.com/msrc/archive/2006/01/13/417431.aspx). *MSRC*. Archived from the original (http://blogs.technet.com/msrc/archive/2006/01/13/417431.aspx) on 2006-03-24. Retrieved 2018-10-27.
4. Litchfield, David. "Bugtraq: Oracle, where are the patches???" (http://seclists.org/lists/bugtraq/2006/May/0039.html). *seclists.org*. Retrieved 2018-10-27.
5. Alexander, Kornbrust. "Bugtraq: RE: Oracle, where are the patches???" (http://seclists.org/lists/bugtraq/2006/May/0045.html). *seclists.org*. Retrieved 2018-10-27.
6. Cerrudo, Cesar. "Bugtraq: Re: [Full-disclosure] RE: Oracle, where are the patches???" (http://seclists.org/lists/bugtraq/2006/May/0083.html). *seclists.org*. Retrieved 2018-10-27.

## External links

- CERT Secure Coding Standards (https://www.securecoding.cert.org/confluence/display/seccode/SEI+CERT+Coding+Standards)